UE24CS341A - Software Engineering
Mini Project

SOFTWARE TEST PLAN

Smart Expense & Budget Management Application

Project	Smart Expense & Budget Management Application
Document	Software Test Plan (STP)
Based On	Software Requirements Specification (SRS) v1.0, dated 04-09-2026
Version	1.0
Authors	Bhuvigna Reddy A T, Bijjala Sai Vishnu Rohit, Blason Raj, Bollavaram Santosh Kumar
SRN	PES2UG24CS119, PES2UG24CS120, PES2UG24CS122, PES2UG24CS123
Status	Draft for Review / Submission

1. Introduction

1.1 Purpose

This Software Test Plan (STP) defines the scope, approach, resources, and schedule for testing the Smart Expense & Budget Management Application. It translates the functional, non-functional, and security requirements defined in the SRS (v1.0) into a verifiable set of test cases so that every requirement (SEB-F-###, SEB-NF-###, SEB-SR-###) is exercised at least once before release.

1.2 Scope

Testing covers user authentication, income and expense recording, transaction management, category management, budget creation and monitoring, alerts, dashboard summaries, analytics, search and filtering, report generation and export, user data isolation, administrative user management, and audit logging. Non-functional testing covers performance, reliability, usability, maintainability, compatibility, and data integrity. Security testing covers HTTPS enforcement, password hashing, authentication/authorization, input validation, and log hygiene.

Out of scope: banking transactions, payment processing, lending, investment execution, and direct bank-account integrations, consistent with the SRS scope statement.

1.3 References

- Software Requirements Specification (SRS) — Smart Expense & Budget Management Application, v1.0, 04-09-2026
- Requirements Traceability Matrix (RTM), Section 8 of the SRS
- UML Use-Case Diagrams — User and Administrator functions, Section 7 of the SRS

2. Test Items

The following modules/components, as defined in the SRS product functions and interfaces, are subject to test:

- Authentication Module (registration, login, password hashing)
- Transaction Module (income/expense add, edit, delete)
- Category Module
- Budget Module (creation and monitoring)
- Notification / Alert Module
- Dashboard Module
- Analytics / Reporting Module
- Search and Filter functionality
- Authorization Layer (user data isolation, role-based access)
- Administration Module
- Audit Logging subsystem
- Web UI (login, dashboard, transaction forms, budget screens, analytics, admin screens)

3. Features to be Tested / Not to be Tested

3.1 Features to be Tested

All 18 functional requirements (SEB-F-001 to SEB-F-018), all 6 non-functional requirements (SEB-NF-001 to SEB-NF-006), and all 5 security requirements (SEB-SR-001 to SEB-SR-005) listed in the SRS are in scope, as detailed in Sections 6-8 of this plan.

3.2 Features Not to be Tested

- Banking transaction processing, payment gateways, lending, and investment execution (explicitly out of scope in the SRS).
- Direct bank-account integrations not present in the current version.
- Third-party email/notification service internals beyond the interface contract.

4. Test Approach and Levels

4.1 Test Levels

- Unit Testing — individual functions/components (e.g., budget calculation, password hashing) validated by developers.
- Integration Testing — module interactions, e.g., Transaction Module feeding Budget Monitoring and Dashboard Summary.
- System Testing — end-to-end verification of the application against the functional, non-functional, and security requirements in this plan.
- User Acceptance Testing (UAT) — representative users validate core workflows (add expense, create budget) per SEB-NF-003.

4.2 Test Types

- Functional testing — Sections 6 test cases (TC-AUTH-*, TC-TXN-*, TC-CAT-*, TC-BUD-*, TC-ALT-*, TC-DASH-*, TC-ANL-*, TC-SRCH-*, TC-RPT-*, TC-SEC-02, TC-ADM-*, TC-AUD-*).
- Non-functional testing — performance, reliability, usability, maintainability, compatibility, data integrity (Section 7).
- Security testing — HTTPS enforcement, password hashing, authN/authZ, input validation, log hygiene (Section 8).
- Regression testing — re-run of the applicable subset of this suite after defect fixes or code changes.

5. Test Environment

Component	Detail
Client	Modern desktop and mobile web browsers (Chrome, Firefox, or equivalent approved browsers)
Backend	Python runtime with Django framework (or equivalent web application stack)
Database	Relational database — MySQL or PostgreSQL, isolated test schema with seeded test data
Network	HTTPS-based test environment mirroring production communication requirements
Notification	Test/sandbox email or notification service for alert verification
Test Data	Seeded test users (standard and administrator roles), sample transactions, categories, and budgets

6. Functional Test Cases

Each test case below maps directly to a functional requirement and its acceptance criterion defined in Section 4 of the SRS.

Req ID	Module	Test Case ID	Objective	Preconditions	Test Steps	Expected Result	Priority
SEB-F-001	Authentication	**TC-AUTH-01**	Verify new user registration with valid and duplicate details	Registration page is accessible	1) Enter valid identity/credential details and submit. 2) Repeat registration with an already-used identifier.	A valid account is created on first attempt; duplicate identifier is rejected with an error.	High
SEB-F-002	Authentication	**TC-AUTH-02**	Verify login with valid and invalid credentials	A registered user account exists	1) Log in with correct credentials. 2) Log in with incorrect credentials.	Valid credentials create an authenticated session; invalid credentials are rejected.	High
SEB-F-003	Authentication	**TC-SEC-01**	Verify passwords are stored using secure hashing	User account created	Inspect the stored password value in the database/user store.	Password is stored as a hash; plaintext password is never stored or displayed.	High
SEB-F-004	Transactions	**TC-TXN-01**	Verify income transaction recording	User is authenticated	Add an income transaction with amount, date, category, and optional description.	Transaction appears in history and totals update correctly.	High
SEB-F-005	Transactions	**TC-TXN-02**	Verify expense transaction recording	User is authenticated	Add an expense transaction with amount, date, category, and optional description.	Transaction is stored and reflected in dashboard/summary totals.	High
SEB-F-006	Transactions	**TC-TXN-03**	Verify transaction update and deletion	At least one transaction exists	1) Edit an existing transaction's values. 2) Delete a transaction.	Updated values are reflected correctly; deleted transaction no longer affects totals.	High
SEB-F-007	Category	**TC-CAT-01**	Verify category create, view, update, delete	User is authenticated	Create, view, update, and delete a custom category.	Permitted category CRUD operations succeed and are reflected in the UI.	Medium
SEB-F-008	Budget	**TC-BUD-01**	Verify budget creation for a category/period	At least one category exists	Create a budget with a positive limit for a selected category and period.	Budget with a positive limit is saved and displayed correctly.	High
SEB-F-009	Budget	**TC-BUD-02**	Verify budget monitoring against actual spending	A budget and related expenses exist	Add, edit, and delete expenses under a budgeted category.	Spending progress against the budget updates correctly after each change.	High
SEB-F-010	Notifications	**TC-ALT-01**	Verify budget threshold/overspend alerts	A budget with a configured threshold exists	Add expenses so spending reaches the warning threshold, then exceeds the budget.	A visible notification is generated at threshold and at exceeded conditions.	High
SEB-F-011	Dashboard	**TC-DASH-01**	Verify dashboard summary accuracy	Transactions and budgets exist for the user	Load the dashboard and compare displayed totals to stored transaction data.	Dashboard totals (income, expenses, balance, recent transactions, budget status) match stored data.	High
SEB-F-012	Analytics	**TC-ANL-01**	Verify category-wise and period-wise analytics	Transactions across multiple categories/periods exist	Apply category and period filters on the analytics view.	Selected filters produce correctly aggregated values in charts/summaries.	Medium
SEB-F-013	Search/Filter	**TC-SRCH-01**	Verify transaction search and filtering	Multiple transactions exist	Search/filter transactions by date range, category, and type.	Returned results match the selected filter criteria exactly.	Medium
SEB-F-014	Reporting	**TC-RPT-01**	Verify report generation for month/custom range	Transactions exist within the target period	Generate a report for a selected month and for a custom date range.	Report totals equal the underlying transaction records for the period.	High

SEB-F-015	Reporting	**TC-RPT-02**	Verify report export	A generated report is available	Export the report in a supported portable format and open it.	Exported file opens successfully and matches the selected report data.	Medium
SEB-F-016	Authorization	**TC-SEC-02**	Verify user data isolation	Two distinct user accounts with data exist	Attempt direct and indirect access to another user's transactions/budgets/reports.	All access attempts to another user's records are denied.	High
SEB-F-017	Administration	**TC-ADM-01**	Verify administrative user management access control	Administrator and standard user accounts exist	1) Perform user management actions as an administrator. 2) Attempt the same actions as a standard user.	Authorized administrator actions succeed; standard users cannot access admin functions.	Medium
SEB-F-018	Audit	**TC-AUD-01**	Verify audit logging of security/admin events	A test action that should be logged is available	Perform a security-relevant or administrative action, then review the audit log.	A retrievable audit entry with timestamp is created without exposing sensitive values.	Medium

7. Non-Functional Test Cases

Each test case below maps to a non-functional requirement and its acceptance/measurement criterion defined in Section 5 of the SRS.

Req ID	Module	Test Case ID	Objective	Preconditions	Test Steps	Expected Result	Priority
SEB-NF-001	Performance	**TC-PERF-01**	Verify dashboard/transaction response time under load	Load-testing tool configured for normal academic deployment load	Execute standard dashboard and transaction operations under simulated concurrent load.	90th percentile response time is ≤ 3 seconds.	High
SEB-NF-002	Reliability	**TC-REL-01**	Verify recovery from transient failure without data loss	A committed transaction exists	Simulate a controlled restart/transient failure and verify data after recovery.	No committed transaction data is lost; the application recovers gracefully.	High
SEB-NF-003	Usability	**TC-UX-01**	Verify navigation clarity and validation messaging	Representative test users available	Have users complete core add-expense and budget-creation tasks; submit invalid form input.	Users complete tasks without critical ambiguity; clear validation messages appear for invalid input.	Medium
SEB-NF-004	Maintainability	**TC-MNT-01**	Verify modular separation of concerns	Source code and architecture documentation available	Perform a code review of presentation, business logic, and data access layers.	Review confirms modular organization and documented configuration.	Medium
SEB-NF-005	Compatibility	**TC-COMP-01**	Verify cross-browser compatibility	Current versions of approved browsers available	Execute core functions (auth, transactions, budgets, dashboard) in each target browser.	Core functions pass in at least Chrome and Firefox or equivalent approved browsers.	Medium
SEB-NF-006	Data Integrity	**TC-DATA-01**	Verify validation of amounts, dates, ownership, required fields	Data entry forms accessible	Submit invalid numeric amounts, invalid dates, mismatched ownership, and missing required fields.	Invalid/inconsistent data is rejected; valid data preserves referential integrity.	High

8. Security Test Cases

Each test case below maps to a security requirement defined in Section 5.1.2 of the SRS.

Req ID	Module	Test Case ID	Objective	Preconditions	Test Steps	Expected Result	Priority
SEB-SR-001	Security	**TC-SEC-03**	Verify HTTPS enforcement for production traffic	Production-equivalent deployment available	Attempt to access the application over plain HTTP.	HTTP access is redirected to HTTPS or rejected per deployment policy.	High
SEB-SR-002	Security	**TC-SEC-01**	Verify approved secure password hashing mechanism	User account created	Inspect password storage implementation and stored values.	Password storage inspection confirms no plaintext passwords; an approved hashing mechanism is used.	High
SEB-SR-003	Security	**TC-SEC-02**	Verify authentication/authorization checks on protected resources	Protected endpoints identified	Attempt to access protected user and admin resources without valid authentication/authorization.	Unauthorized requests are denied for all protected resources.	High
SEB-SR-004	Security	**TC-SEC-04**	Verify input validation and sanitization	Input fields accepting user-controlled data identified	Submit malformed and malicious inputs (e.g., injection payloads) into forms and API fields.	Malformed and malicious test inputs do not alter unintended data or execution.	High
SEB-SR-005	Security	**TC-SEC-05**	Verify sensitive values are not written to logs	Logging enabled in test environment	Perform login and other actions that handle credentials, then inspect application logs.	Log inspection confirms passwords, authentication secrets, and sensitive credentials are absent.	High

9. Entry and Exit Criteria

9.1 Entry Criteria

- SRS is approved/baselined and this Test Plan is reviewed.
- Test environment is provisioned and seeded with required test data.
- Build under test is deployed and smoke-tested (application is reachable and core pages load).

9.2 Exit Criteria

- All high-priority functional requirements are implemented and verified (per SRS Section 6.2).
- All mandatory security requirements (SEB-SR-001 to SEB-SR-005) pass their defined tests.
- No critical or high-severity defects remain open for core workflows.
- Non-functional acceptance targets are met, or approved deviations are documented.
- The Requirements Traceability Matrix shows full coverage from requirements to test cases and results.

10. Suspension and Resumption Criteria

- Suspend testing if a blocking defect prevents execution of a core workflow (e.g., login failure, data corruption).
- Suspend testing if the test environment becomes unavailable or test data integrity is compromised.
- Resume testing once the blocking defect is fixed, verified in a smoke test, and the environment is stable.

11. Roles and Responsibilities

Role	Responsibility
Project Team / Developers	Implement fixes, support unit and integration testing, provide build releases
Test Engineers	Design, execute, and log results for test cases in this plan; report defects
Project Guide / Reviewer	Review test plan and results; approve exit from test cycles
Administrator (test role)	Support administrative and audit-logging test cases

12. Deliverables

- This Software Test Plan document
- Executed test case results with pass/fail status per test case ID
- Defect log with severity, status, and resolution notes
- Updated Requirements Traceability Matrix (Status column populated per SRS Section 8)
- Test summary report at the end of each test cycle

13. Risks and Mitigation

Risk	Mitigation
Incomplete test data leading to false negatives	Seed a standardized, version-controlled test dataset before each cycle
Environment drift between test and production configuration	Mirror production-equivalent configuration in the test environment
Limited time for full regression before submission	Prioritize High-priority requirements and security tests first
Notification/email service unavailability during alert testing	Use a sandbox/mock notification service